

Activity 3:

To implement the **Alpha-Beta Pruning algorithm** in Python and demonstrate how it optimizes the **Minimax search algorithm** by reducing the number of nodes explored in a two-player game tree.

Theory

In two-player games such as chess, tic-tac-toe, or checkers, an intelligent agent must decide its next move by anticipating the opponent's responses.

The **Minimax algorithm** is a decision-making algorithm used to minimize the possible loss in a worst-case scenario.

However, the **Minimax** algorithm explores all nodes in the game tree, which is computationally expensive.

To overcome this, we use **Alpha-Beta Pruning**, an optimization technique that eliminates branches that do not influence the final decision, thereby **reducing computation time**.

Algorithm:

1. Start from the root node (Maximizer's turn).
2. Initialize $\alpha = -\infty$ and $\beta = +\infty$.
3. Recursively evaluate the game tree using Minimax logic:
 - If it's the maximizer's turn, choose the maximum value and update α .
 - If it's the minimizer's turn, choose the minimum value and update β .
4. If at any point $\beta \leq \alpha$, **prune** the remaining branches.
5. Continue until reaching the leaf nodes (terminal states).

Code:

```
# Activity 3: Minimax Algorithm with Alpha-Beta Pruning
```

```
# Name: Arun M.
```

```
# Department: Computer Science and Engineering
```

```
# Year: Final Year
```

```
# Counters for comparison
```

```
nodes_visited_minimax = 0
```

```
nodes_visited_alphabeta = 0
```

```
# Normal Minimax Implementation
```

```
def minimax(depth, node_index, maximizing_player, values, max_depth):
```

```
    global nodes_visited_minimax
```

```
    nodes_visited_minimax += 1
```

```
    # Base case
```

```
    if depth == max_depth:
```

```
        return values[node_index]
```

```
    if maximizing_player:
```

```
        return max(
```

```
            minimax(depth + 1, node_index * 2, False, values, max_depth),
```

```
            minimax(depth + 1, node_index * 2 + 1, False, values, max_depth),
```

```
        )
```

```
    else:
```

```
        return min(
```

```
            minimax(depth + 1, node_index * 2, True, values, max_depth),
```

```
            minimax(depth + 1, node_index * 2 + 1, True, values, max_depth),
```

```
        )
```

```

# Minimax with Alpha-Beta Pruning

def alphabeta(depth, node_index, maximizing_player, values, max_depth, alpha, beta):

    global nodes_visited_alphabeta

    nodes_visited_alphabeta += 1

    # Base case

    if depth == max_depth:

        return values[node_index]

    if maximizing_player:

        best = float("-inf")

        for i in range(2):

            val = alphabeta(depth + 1, node_index * 2 + i, False, values, max_depth, alpha, beta)

            best = max(best, val)

            alpha = max(alpha, best)

            if beta <= alpha:

                break # Prune remaining branches

        return best

    else:

        best = float("inf")

        for i in range(2):

            val = alphabeta(depth + 1, node_index * 2 + i, True, values, max_depth, alpha, beta)

            best = min(best, val)

            beta = min(beta, best)

            if beta <= alpha:

```

```
        break # Prune remaining branches

    return best
```

```
# Example Game Tree (leaf node values)
```

```
values = [3, 5, 6, 9, 1, 2, 0, -1]
```

```
max_depth = 3
```

```
# Run algorithms
```

```
best_value_minimax = minimax(0, 0, True, values, max_depth)
```

```
best_value_alphabeta = alphabeta(0, 0, True, values, max_depth, float("-inf"), float("inf"))
```

```
# Display Results
```

```
print("=====")
```

```
print("  Results of Minimax vs Alpha-Beta Pruning  ")
```

```
print("=====")
```

```
print(f"Leaf Node Values: {values}")
```

```
print(f"Best Value (Normal Minimax): {best_value_minimax}")
```

```
print(f"Best Value (Alpha-Beta Pruning): {best_value_alphabeta}")
```

```
print("-----")
```

```
print(f"Nodes explored in Minimax: {nodes_visited_minimax}")
```

```
print(f"Nodes explored in Alpha-Beta Pruning: {nodes_visited_alphabeta}")
```

```
print("-----")
```

```
print(f"Pruning saved: {nodes_visited_minimax - nodes_visited_alphabeta} nodes!")
```

Output:

=====

Results of Minimax vs Alpha-Beta Pruning

=====

Leaf Node Values: [3, 5, 6, 9, 1, 2, 0, -1]

Best Value (Normal Minimax): 5

Best Value (Alpha-Beta Pruning): 5

Nodes explored in Minimax: 15

Nodes explored in Alpha-Beta Pruning: 11

Pruning saved: 4 nodes!